

Fore-and-Back for Changepoint Analysis

Vittorio Maniezzo¹[0000–0002–1220–1235]

University of Bologna, Cesena, Italy
`vittorio.maniezzo@unibo.it`
<https://www.unibo.it/sitoweb/vittorio.maniezzo/en>

Abstract. The abstract should briefly summarize the contents of the paper in 150–250 words.

Keywords: Changepoint analysis · matheuristics · Fore-and-Back.

1 Introduction

Please note that the first paragraph of a section or subsection is not indented. The first paragraph that follows a table, figure, equation etc. does not need an indent, either.

Subsequent paragraphs, however, are indented.

Sample Heading (Third Level) Only two levels of headings should be numbered. Lower level headings remain unnumbered; they are formatted as run-in headings.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable.

2 Changepoint analysis

Letteratura etc. [3]

3 Fore-and-Back for changepoint analysis

Fore-and-back [1, 3] is an extension of *Beam Search* (BS) that can improve its effectiveness. Fore-and-Back, when run with no limits on computational resources, becomes an exact solution method. However, by design, it is mainly concerned with heuristic solving, trying to quickly get high quality solutions with little attention paid to optimality proofs.

A significant characteristic of this method is that, despite being a primal only method, it is able to compute bounds to the cost of completing partial solutions, therefore to discard partial solutions from expansion and ultimately to reduce the search space.

Algorithm 1: Algorithm Fore-and-Back

```

1 function Fore-and-Back( $\delta$ ,  $maxn$ ,  $maxnodes$ );
   Input :  $\delta$ , beam width,  $maxn$ , max total num of nodes,  $maxnodes$ , max
           num of nodes per tree
   Output: A feasible solution  $\mathbf{x}^*$  of value  $z_{best}$ 
2 initialize  $t = 0$ ,  $noimpr = 0$ ,  $nNodes = 0$ ,  $z_{best} = \infty$ ;
3 while  $nNodes \leq maxn$  do // Alternate forward and backward trees
4    $t = t + 1$ ;  $noimpr = noimpr + 1$ ;
5   let  $L_0^t = L_{n+1}^t = \{\emptyset\}$ ,  $ntNodes = 0$ ;
6   foreach level  $h = 1, \dots, n$  do
7     set  $T_h = \{\emptyset\}$ ; // generate the node set  $T_h$ 
8     if  $t$  is odd then
9       | set  $k = h$ ;  $k1 = h - 1$ ;
10    else
11      | set  $k = n - h + 1$ ;  $k1 = n - h + 2$ ;
12    end
13    foreach node  $X \in L_{k-1}^t$  do // Possible expansions
14      foreach component  $s \in S_k$  do
15        let  $X' = X \cup \{s\}$ ;
16        if  $X'$  meets conditions  $i)$  and  $ii)$  then
17          | set  $T_k = T_k \cup X'$ ;
18          |  $ntNodes = ntNodes + 1$ ;
19          |  $nNodes = nNodes + 1$ ;
20          if  $h = n$  and  $c(X') < z_{best}$  then // feasible solution
21            | set  $z_{best} = c(X')$ ,  $noimpr = 0$  and  $\mathbf{x}^* = X'$ ;
22          end
23        end
24      end
25      set  $E^k = E^k \cup X$ ; // actual node expansion
26    end
27    foreach node  $X \in T_k$  do // extract the subset  $L_k^t \in T_k$ 
28      if  $|T_k| \leq \delta$  then
29        | set  $L_k^t = T_k$ ;
30      else
31        | let  $L_k^t$  contain only the  $\delta$  least cost partial solutions of  $T_k$ ;
32      end
33    end
34    if  $ntNodes > maxnodes$  then break;
35  end
36  if  $noimpr = 2$  then stop ; // sol. not improved for two iterations
37 end

```

Beam Search [2]

In C++, the `std::map<int, int>` is an excellent choice because it is implemented as a Red-Black tree, which keeps the keys (your num values) sorted. Store the pairs as `std::map<int, int> map name;` where the key is num and the value is the minimum cost seen so far for that specific num and all numbers greater than it up to the next existing entry. However, a simpler, more robust approach is to ensure the map only stores non-redundant pairs that form the Pareto frontier of the $(num, cost)$ space. Operation Standard BBST - `std::mapBBST` with Pareto Optimization Insert $O(\log N)$ $O(\log N)$ amortized (because dominated elements are permanently removed) Query $O(\log N)$ $O(\log N)$ (for upper bound and decrement)

4 Computational results

Computational results

Sample Heading (Fourth Level) The contribution should contain no more than four levels of headings. Table 1 gives a summary of all heading levels.

Table 1. Table captions should be placed above the tables.

Heading level	Example	Font size and style
Title (centered)	Lecture Notes	14 point, bold
1st-level heading	1 Introduction	12 point, bold
2nd-level heading	2.1 Printing Area	10 point, bold
3rd-level heading	Run-in Heading in Bold. Text follows	10 point, bold
4th-level heading	<i>Lowest Level Heading.</i> Text follows	10 point, italic

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead (see Fig. 1).

5 Conclusions

The conclusions

References

1. Bartolini, E., Maniezzo, V., Mingozzi, A.: An adaptive memory-based approach based on partial enumeration. In: Maniezzo, V., Battiti, R., Watson, J.P. (eds.) LION 2, LNCS 5313, pp. 12–24. Springer (2008)
2. Blum, C.: Beam-ACO for simple assembly line balancing. *INFORMS Journal on Computing* **20**(4), 618–627 (2008)
3. Maniezzo, V., Boschetti, M.A., Stützle, T.: *Matheuristics: Algorithms and Implementations*. EURO Advanced Tutorials on Operations Research, Springer (2021). <https://doi.org/10.1007/978-3-030-64219-4>

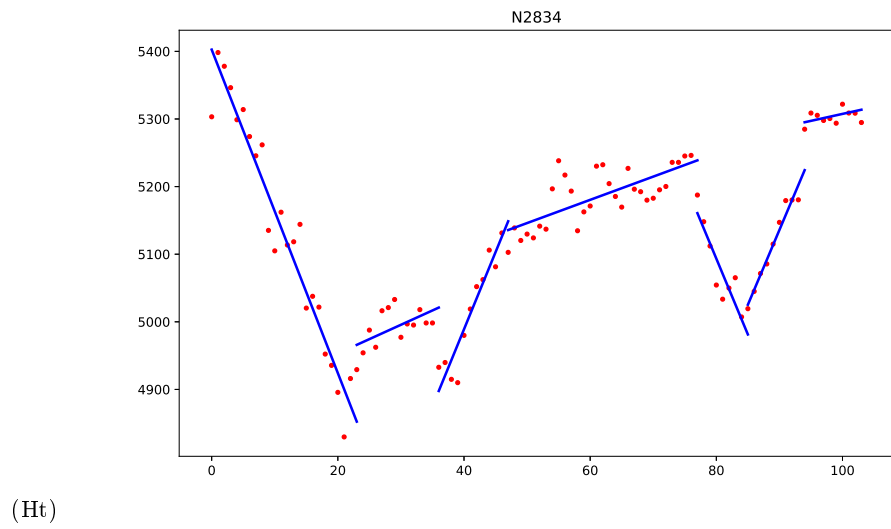


Fig. 1. A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.